LAB Program - 2.

WAP to convert a given valid parantheised infix arithmetic exp to postfix expresion. The exp consists of single character operands & the binary operators +, -, *, /.

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAX 100
char stack[MAX];
int top = -1;

void push (char c){
    if (top == MAX - 1){
        printf ("stack Overflow \n");
        return;
    }
    stack [++top] = c;
}

char pop (){
    if (top == -1){
        printf ("stack Underflow \n");
        return -1;
    }
    return stack [top--];
}

char peek (){
    if (top == -1){
        printf ("stack Overflow \n");
        return -1;
    }
```

```
    return stack[top];
}

int precedence (char op){
    switch (op){
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        case '(':
            return 0;
        default:
            return -1;
    }
}

//0 = Left-to-Right , 1 = Right-to-Left
int associativity (char op){
    if (op == '^'){
        return 1;
    }
    return 0;
}

void InfixtoPostfix (char infix[], char postfix[]){
    int i, k=0;
    char c;
    for (i=0; infix[i] != '\0'; i++){
        c = infix[i];
        if
```

```
        if(isalnum(c)){
            postfix[k++] = c;
        }
        else if (c == '('){
            push(c);
        }
        else if (c == ')'){
            while (peek() != '('){
                postfix[k++] = pop();
            }
            pop;
        }
        else{
            while (top != -1 && ((precedence(peek()) >
                    precedence (c)) ||
                (precedence(peek()) == precedence(c) &&
                    anociativity(c) == 0))){
                postfix[k++] = pop();
            }
            push(c);
        }
    }
    while (top != -1){
        postfix[k++] = pop();
    }
    postfix[k] = '\0';
}

int main(){
    char infix[MAX], postfix[MAX];
    printf("Enter a valid paranthesis infix expression:");
    scanf("%s", infix);
    InfixtoPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
```

Date __/__/__
Page _____

return 0;
}

Output:
Enter a valid parathesis infix expression:
(A+(B*C-(D/E^F)*G)*H)

Postfix expression:
ABC*DEF^/G*-H*+.

13/10